Tomasulo Multi-Issue Simulator

Programming assignment report

Ahmed Ahmed | October 14, 2026

44 tests passed **28 benchmark runs passed**

What I implemented

I extended and reorganized the supplied Python simulator to issue up to 1, 2, 3, or 4 instructions in program order each cycle. Issue width defaults to 1 and can be set in the input file or overridden on the command line. Issue stops at the first instruction that cannot obtain the required resources.

I capture each instruction's source operands before assigning its destination a new ROB tag. This makes dependencies inside the same issue group work correctly, including RAW, WAR, and WAW cases. Execution can proceed out of order, while architectural register updates and stores retire in order.

Module	Main change
parser.py / model.py	Read hardware configuration, initial values, and all required instructions; validate input.
engine.py	Implement multi-issue, RAT/ROB scheduling, one CDB, speculative branches, load/store forwarding, and memory replay.
main.py / output.py	Print cycle tables, all registers, and nonzero memory; optionally save full JSON traces.
tests/ / run_experiments.py	Check timing rules and compare final state with an independent sequential interpreter.

Timing choices

Only issue bandwidth is widened. The simulator keeps one CDB and one in-order commit slot. Integer and address units are unpipelined; FP units are pipelined. Stage intervals are inclusive, and a result becomes usable in the following cycle. For a one-cycle Addi: ISSUE 1, EX 2, WB 3, COMMIT 4. Stores write memory only while committing.

The 8-entry tagged BTB uses a one-bit predictor. A misprediction detected at the end of cycle n squashes younger work, restores the RAT, and resumes correct fetch in n+2. A load that bypasses an unresolved older store is replayed if that store later reveals a conflicting address.

Results across issue widths

Each row uses the same hardware configuration for all four widths. Values are total simulated cycles; lower is better. Every run matched the independent interpreter for all registers and memory.

Benchmark	Width 1	Width 2	Width 3	Width 4	W4 speedup
Corrected starter loop	66	65	65	65	1.02x
Independent mixed operations	47	48	47	47	1.00x
Dependency / renaming chain	40	38	38	36	1.11x
Store-to-load forwarding	44	44	44	44	1.00x
Branch recovery	44	44	44	44	1.00x
Memory disambiguation / replay	34	34	35	35	0.97x
Small ROB / RS / LSQ	42	42	42	42	1.00x

Speedup = cycles at width 1 / cycles at width 4. A value below 1.00x means width 4 took more cycles. Full ISSUE, EX, MEM, WB, and COMMIT traces are included in results/.

What the results show

The dependency and renaming benchmark improves from 40 cycles at width 1 to 36 at width 4: 10% fewer cycles, or a 1.11x speedup. Wider issue exposes the FP chain earlier while the integer chain is still running. The corrected starter improves from 66 to 65 cycles.

Several tests show little or no improvement because issue is only one limit. The single CDB, serialized memory service, true dependencies, and small resource queues still constrain progress. Wider issue can also change contention and replay timing: the memory-replay test takes 35 cycles at width 4 versus 34 at width 1. More issue slots do not guarantee fewer total cycles.

Selected correctness results

Test	Final values / checks at all four widths
Corrected starter loop	R1=0, R2=20; F2=3, F4=9, F6=15, F20=6. Mem[24]=15, Mem[28]=11, Mem[32]=7.
Forwarding	F4=7, F5=7; Mem[0]=4, Mem[4]=7, Mem[8]=11. Two loads forward from older stores in each run.
Branch recovery	17 instructions commit; R3=108, R4=112, R5=113. Wrong-path store is discarded: Mem[0] remains 10.
Memory replay	One replay in each run; F2=9 and F3=18. Final Mem[16]=9 and Mem[20]=18.

Testing and reproduction

How I checked correctness

I ran 44 test methods successfully using Python 3.12.14. The suite includes 72 seeded differential configurations: 10 straight-line programs and 8 bounded loop programs, each run at issue widths 1-4 with varied hardware settings. The oracle has its own assembly parser and models architectural execution independently of the simulator.

Separate deterministic checks cover same-group dependencies, stale RAT updates, hardwired R0, ordered issue stalls, ROB/RS/LSQ capacity, FP pipelining, integer serialization, one CDB, one commit per cycle, forwarding from the youngest matching store, unavailable store data, memory replay, BTB collisions, branch recovery, invalid input, precise faults, and command-line output.

Example: width-4 dependency trace

These four instructions all issue in cycle 1. Their RAW dependencies still force execution to wait for the correct producer; issuing together does not mean executing together.

Instruction	ISSUE	EX	WB	COMMIT
Add R3 R1 R2	1	2-3	4	5
Addi R1 R3 1	1	5-6	7	8
Sub R4 R1 R2	1	8-9	10	11
Add R3 R4 R1	1	11-12	13	14

Correction to the supplied input

The original test case uses branch offset -12. The assignment defines branch targets as PC+4+(offset<<2), so this offset leaves the program. I preserved the original and supplied a corrected version with offset -8, which repeats the full eight-instruction loop. The corrected loop commits 25 dynamic instructions.

Reproduce the results

```
python3 -m unittest discover -s tests -v
python3 run_experiments.py
python3 main.py benchmarks/corrected_starter.in --issue-width 4
```

The submission includes runnable source, the original starter for comparison, test inputs, the independent oracle, a README, the test log, summaries, and all 28 text/JSON simulation traces. No external Python packages are needed to run the simulator or tests.

Scope and limitations

No known failing cases remain in the included tests. This teaching model uses Python floating-point arithmetic, rather than bit-accurate ISA rounding. Memory is interpreted as 64 aligned words at byte addresses 0-252; unaligned access faults. Forwarding shares the memory-stage service, and a store records its commit at the end of its memory interval. These choices are documented in the README.